

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

A Privacy-Preserving Edge Gateway for Real-Time Audio Deepfake Detection Using ARM TrustZone Confidential Inference

By
Waleed Asif

Supervisor: **Dr. Owen Foley**

October 25, 2026

M.Sc. in Computing

Masters Thesis

Submitted in fulfilment of the requirements for the award of **Master of Science in Computing** to the Department of Computer Science & Applied Physics, Atlantic Technological University (ATU), Galway.

Contents

1 Introduction **1**
- 1.1 Context . . . 1
- 1.2 Objectives . . . 1
- 1.3 Scope . . . 2
- 1.4 Thesis Structure . . . 2

2 Research Methodology **4**
- 2.1 Software Development Methodology . . . 4
- 2.2 Research Methodology . . . 5
  - 2.2.1 Verification-driven development . . . 5
  - 2.2.2 Controlled experimental elimination . . . 6
  - 2.2.3 Threats to validity . . . 6

3 Literature Review **7**
- 3.1 Audio Anti-Spoofing Architectures . . . 7
- 3.2 Privacy-Preserving On-Device Inference . . . 8
- 3.3 From Laboratory Benchmarks to the Telephony Edge . . . 9
- 3.4 Summary and Positioning . . . 10

4 System Design **12**
- 4.1 Detection Backbone . . . 12
- 4.2 Trusted Execution Architecture . . . 12
  - 4.2.1 Design decision: emulation platform . . . 14
- 4.3 Model Deployment Pipeline . . . 14
  - 4.3.1 Design decision: ONNX-to-C code generation tool . . . 14
- 4.4 Resolving the Secure-World Memory Constraint . . . 15
- 4.5 End-to-End System Architecture . . . 16
- 4.6 Live Telephony Integration Design . . . 17

5 System Evaluation **19**
- 5.1 Verification of Objective O2: Correctness of Confidential Inference . 19
- 5.2 Diagnosing the Secure-World Memory Constraint . . . 20
  - 5.2.1 The failure . . . 20
  - 5.2.2 Elimination and measurement . . . 21
  - 5.2.3 A self-corrected diagnostic error . . . 22
- 5.3 Domain-Generalisation Investigation . . . 22
- 5.4 Live Telephony Integration Status . . . 23
- 5.5 Limitations and Threats to Validity . . . 24

6 Conclusion **25**
- 6.1 Future Work . . . 26

A Source Code Repository **29**

List of Figures

4.1 RawTFNet-16 architecture, used unmodified as the detection backbone. The frontend/classifier boundary (green/orange) is the seam used for secure-world partitioning in Section 4.4. 13
4.2 The trusted application / client application session pattern used by OP-TEE. This pattern was validated with a minimal official example (Milestone M2, Table 2.1) before any model-specific code was written. 13
4.3 Model deployment pipeline. Correctness is checked numerically at every stage so that any discrepancy is caught at the stage that introduced it, rather than after secure-world integration. 15
4.4 Partitioned deployment. Only the classifier's forward pass executes inside the secure world; frontend feature extraction remains in the normal world for this proof-of-concept, a scope boundary discussed further in Section 5.5. 16
4.5 End-to-end system architecture. 17

List of Tables

2.1 Milestone-based development structure. 5

3.1 Research questions and where each is addressed in this thesis. . . . 11

5.1 Verification evidence at every stage of the deployment pipeline. . . . 20
5.2 Diagnostic sequence leading to the partitioned design. 21
5.3 Domain-generalisation elimination study: candidate causes of real-world misclassification. 23
5.4 Limitations and threats to validity. 24

Abstract

Residential telephony has become a primary channel for voice-based fraud enabled by generative speech synthesis, yet existing anti-spoofing research is evaluated almost entirely on clean laboratory benchmarks and rarely considers how detection should be deployed at the network edge without exposing a caller's voice, or the detection model itself, to a compromised host. This thesis addresses that gap by designing and implementing a proof-of-concept privacy-preserving edge gateway for real-time audio deepfake detection, combining a lightweight anti-spoofing network (RawTFNet-16) with confidential inference inside an ARM TrustZone trusted execution environment (OP-TEE).

The trained model was exported through a verified PyTorch $\rightarrow$ ONNX $\rightarrow$ C pipeline and embedded in a signed Trusted Application. Because the compiled model's working memory requirement (319.5 MB) exceeded the secure-world memory pool available on the target emulation platform (approximately 16 MB), the network was partitioned at its existing front-end/classifier boundary so that only the classifier executes inside the secure world. The partitioned pipeline was verified end-to-end on a QEMU ARMv8-A emulation of the OP-TEE trusted execution environment, reproducing the reference PyTorch model's output to six decimal places and providing tamper-resistant, secure-world-logged evidence of which audio was processed. A controlled elimination study was then used to investigate why the model, trained on the ASVspoof corpus, consistently misclassified real-world recordings: sample rate, loudness normalisation, speaker accent, and background noise floor were tested individually, isolating the residual gap to a genuine domain-generalisation limitation rather than a fixable preprocessing artefact. A live-call integration with an Asterisk private branch exchange was designed and partially implemented, using a four-second sliding window matched to the network's duration sensitivity.

The results demonstrate that confidential inference for audio anti-spoofing is achievable end to end on TrustZone-class hardware, and characterise, for the first time in this deployment context, the extent to which laboratory-trained detection accuracy fails to transfer to uncontrolled recording conditions.

Acknowledgments

I would like to thank my supervisor, Dr. Owen Foley, for his guidance, feedback, and patience throughout this project, including his willingness to provide independent test recordings for the generalisation study reported in Chapter 5. I am grateful to the Department of Computer Science and Applied Physics at Atlantic Technological University Galway for the facilities and support that made this research possible. Finally, I would like to thank my family for their support and encouragement throughout the course of this MSc.

Chapter 1

Introduction

1.1 Context

Generative speech synthesis has reached a level of realism that makes synthetic voices difficult to distinguish from genuine human speech by ear alone [1]. This capability is increasingly weaponised in residential telephony fraud: a caller's voice can be cloned from a few seconds of publicly available audio and used to impersonate a family member or an authority figure in a live phone call. Two properties of this threat make it a harder engineering problem than laboratory anti-spoofing research typically assumes. First, a usable defence has to produce a decision *while the call is still connected*, using a rolling window of audio, rather than analysing a completed recording after the fact. Second, because the input is a person's voice and the detection model may represent proprietary work, the inference computation itself should be shielded from whatever else is running on the device that hosts it – including a compromised or malicious host operating system.

Existing anti-spoofing research addresses detection accuracy in isolation, typically on the ASVspoof benchmark family under controlled recording conditions, and existing confidential-computing research addresses trusted execution in isolation, typically for generic machine-learning inference rather than for a real anti-spoofing network under memory-constrained edge conditions. No peer-reviewed work combines a lightweight anti-spoofing detector with ARM TrustZone-based confidential inference specifically for voice deepfake detection at the telephony edge. This thesis is motivated by that gap.

1.2 Objectives

The overarching goal of this research is to design and demonstrate a privacy-preserving edge gateway for real-time audio deepfake detection in residential tele-

phony. This thesis reports on the confidential-inference proof-of-concept that underpins that gateway, and pursues three specific objectives, each corresponding to one of the research questions developed in the literature review (Chapter 3):

O1. Select and justify a detection architecture suitable for edge deployment, balancing detection accuracy (equal error rate, EER) against the computational and memory footprint that a resource-constrained trusted execution environment can support.

O2. Design and implement a confidential-inference pipeline that runs the selected detector's forward pass inside an ARM TrustZone secure world, verify its numerical correctness against the original trained model at every stage of the deployment pipeline, and produce independent, tamper-resistant evidence that inference genuinely executed inside the secure world.

O3. Characterise the system's behaviour under conditions representative of a real telephony deployment – both the audio domain-generalisation gap between laboratory training data and real-world recordings, and the practical constraints of integrating the pipeline with a live call through a private branch exchange.

1.3 Scope

No physical Raspberry Pi 4 – the intended deployment target – was available during this phase of the work. The confidential-inference component reported here was therefore built and verified on an emulated ARMv8-A platform (QEMU, running OP-TEE), a substitution that is examined explicitly in Section 4.2.1. The full thesis scope additionally includes combining federated learning and differential privacy with confidential inference to give end-to-end privacy guarantees across a fleet of gateways; that combination is treated in the literature review as an open research gap (Section 3.2) and is scoped as future work rather than implemented in this phase, so that the confidential-inference primitive could first be verified as achievable on its own.

1.4 Thesis Structure

Chapter 2 describes the software development and research methodologies used. Chapter 3 reviews the literature underpinning each of the three objectives above and derives the research questions that structure the rest of the thesis. Chapter 4 presents the system design: the detection backbone, the TrustZone/OP-TEE

architecture, the model conversion pipeline, and the model-partitioning solution that made confidential inference feasible under the platform's memory constraints. Chapter 5 evaluates the system against the objectives set out above, reporting verification evidence, the domain-generalisation investigation, and the system's limitations. Chapter 6 concludes the thesis and outlines future work.

Chapter 2

Research Methodology

This project combined two distinct methodological strands: a *software development methodology* governing how the system itself was built, and a *research methodology* governing how empirical claims about the system's correctness and behaviour were established. Both are described below, along with the specific validity threats each was designed to address.

2.1 Software Development Methodology

The confidential-inference pipeline was developed using an iterative, milestone-based approach rather than a single up-front design-and-build cycle. This was a deliberate choice: the target runtime (a trusted execution environment with a small, fixed secure-memory pool and no dynamic machine-learning framework support) was unfamiliar territory, and several of its practical constraints – in particular, the exact size of the available secure-memory pool – could only be discovered empirically rather than read from documentation. An iterative approach allowed each milestone to de-risk a specific unknown before the next milestone depended on it.

Table 2.1 summarises the eight milestones used to structure development. Each milestone was independently demonstrable and produced its own verification evidence (detailed in Chapter 5), so that the project's progress did not depend on every milestone succeeding on the first attempt – and, as Section 5.2 shows, one of them did not.

Each milestone was closed only once it produced independent verification evidence rather than a plausible-looking result – a discipline that directly prevented an incorrect diagnosis from propagating into the system design, as described in Section 5.2.3.

Stage	Purpose
M1	Bring up the OP-TEE / QEMU environment and confirm the toolchain with the official test suite.
M2	Validate the trusted-application / client-application communication pattern with a minimal official example, before any model-specific code was written.
M3	Export the trained model to ONNX and verify its output against the original PyTorch model.
M4	Compile the verified ONNX graph to standalone, dependency-free C suitable for a trusted application.
M5	Link the compiled model into a trusted application and attempt to load and run it.
M6	Diagnose and resolve the resulting memory-capacity failure (Section 5.2).
M7	Generalise the pipeline to arbitrary audio input and add tamper-resistant evidence of secure-world execution.
M8	Integrate the verified pipeline with a live telephony call through Asterisk.

Table 2.1: Milestone-based development structure.

2.2 Research Methodology

Two research questions in this thesis required an empirical, rather than purely constructive, methodology.

2.2.1 Verification-driven development

Every stage of the model deployment pipeline (PyTorch, ONNX, the independent Apache TVM cross-check, the generated C, and finally the in-enclave execution) was checked numerically against the previous stage on held-out audio before being trusted as a foundation for the next stage. This approach was chosen specifically because a trusted execution environment offers very limited debugging visibility once code is running inside it: a numerical discrepancy introduced early in the pipeline but only discovered after secure-world integration would be substantially harder to trace to its source. Verifying at each stage bounds the search space for any future discrepancy to the single stage most recently added.

2.2.2 Controlled experimental elimination

The investigation into why the trained model misclassified real-world audio (Section 5.3) used a controlled elimination methodology: each candidate explanatory variable – sample rate, signal loudness, speaker accent, and background noise floor – was isolated and tested individually, with all other variables held constant, before being accepted, rejected, or fixed. This approach was chosen over an uncontrolled, multi-variable comparison because the goal was not merely to observe that real-world audio misclassifies, but to establish *which*, if any, of the readily fixable preprocessing mismatches were responsible, so that a genuine model-generalisation limitation could be distinguished from a pipeline defect. A same-pipeline, native-speaker control recording was used specifically to weaken the accent hypothesis without conflating it with the confounded channel and recording-device differences present in the original comparison.

2.2.3 Threats to validity

Two threats to the validity of the empirical findings are acknowledged here and expanded upon in Chapter 5. First, all confidential-inference results were obtained on an ARMv8-A software emulation of the target platform rather than physical hardware, so any timing results are not representative of deployed performance (Section 4.2.1). Second, the elimination study in Section 5.3 used a small, opportunistically gathered sample of real-world recordings rather than a large, demographically balanced corpus; its conclusions should therefore be read as characterising the *existence and rough shape* of a generalisation gap rather than providing a precise, generalisable effect size.

Chapter 3

Literature Review

This chapter reviews the literature underpinning the three objectives set out in Chapter 1 and, in doing so, derives the three research questions that structure the remainder of the thesis. Section 3.1 addresses detection architecture selection, Section 3.2 addresses the combination of privacy-preserving techniques for on-device inference, and Section 3.3 addresses the transfer of laboratory detection performance to the telephony edge.

3.1 Audio Anti-Spoofing Architectures

RQ1: Which recent audio deepfake detection architectures best balance detection accuracy against the computational footprint required for real-time inference on resource-constrained edge devices?

Modern anti-spoofing systems have converged on end-to-end architectures that consume the raw audio waveform directly, avoiding hand-crafted front ends such as mel-frequency cepstral coefficients or constant-Q transforms. RawNet2 [2] demonstrated that a raw-waveform convolutional network with a learnable sinc-filter front end could match or exceed spectrogram-based detectors, but at a parameter count (17.6M) that is difficult to justify inside a memory-constrained trusted execution environment. AASIST [3] improved detection accuracy substantially by modelling spectro-temporal relationships with a graph attention mechanism, reducing the parameter count relative to RawNet2 but at a higher computational cost (8.9G multiply-accumulate operations, MACs, per inference).

RawTFNet [1] was selected as the detection backbone for this project because it directly targets the accuracy/footprint trade-off that RQ1 asks about. Its Time-Frequency Convolution design processes the frequency and time axes of the network's internal feature map separately before recombining them, using depthwise-separable convolutions and squeeze-and-excitation attention to keep the parameter

count low. At its smallest published configuration (RawTFNet-16, 0.07M parameters, 2.9G MACs), it reaches a lower equal error rate (4.50%) on the ASVspoof 2021 LA evaluation set than RawNet2, AASIST, or either of the two Rawformer variants reported in the same comparison, while being an order of magnitude smaller than RawNet2 in parameter count. This combination of footprint and accuracy is the basis for its selection in Chapter 4.

A second finding from the RawTFNet literature directly shaped a design decision in this project: detection accuracy is duration-sensitive, degrading sharply on clips under two seconds and recovering only marginally beyond eight seconds, with a minimum equal error rate around the network's own four-second training segment length. This motivated the choice of a four-second sliding window for the live-detection design in Section 4.6, rather than a shorter window that would reduce end-to-end latency.

A recurring theme across this line of work, however, is that reported accuracy figures are obtained on the ASVspoof benchmark family under controlled recording conditions. Müller et al. [4] showed that anti-spoofing detectors trained on ASVspoof-style data generalise poorly to an independently collected corpus of real-world deepfake audio (the *In-the-Wild* dataset), with equal error rates rising substantially relative to in-domain evaluation. This generalisation gap is directly relevant to RQ3 (Section 3.3) and, as Section 5.3 of this thesis reports, was independently reproduced and characterised for RawTFNet during this project's own empirical work.

3.2 Privacy-Preserving On-Device Inference

RQ2: How can trusted execution environments be combined with federated learning and differential privacy to provide end-to-end privacy guarantees for biometric voice data during on-device deepfake detection?

ARM TrustZone provides a hardware-enforced isolated execution mode, and OP-TEE is the dominant open-source trusted operating system exposing that isolation through a standard programming model of trusted applications and client applications. Confidential inference – running a trained model's forward pass inside this isolated environment so that its weights and computation are inaccessible to a compromised host – has been demonstrated for generic convolutional networks. DarkneTZ [5] partitions a convolutional network between an untrusted host and a TrustZone-protected enclave, keeping only the most privacy-sensitive later layers inside the secure world to balance confidentiality against the enclave's limited compute and memory budget. This partitioning strategy is conceptually related to, though independently motivated from, the model-partitioning solution

developed in this thesis (Section 4.4): both split a network to fit a constrained secure-memory budget, but DarkneTZ partitions primarily for confidentiality of late-stage features, whereas this thesis partitions primarily to satisfy a hard memory ceiling while keeping the entire classifier – the part of the network that produces the decision – inside the secure world.

Complementary to confidential inference, federated learning [6] allows a model to be trained across many devices without centralising raw training data, and differential privacy [7] bounds how much any single training example can influence a model's learned parameters, providing a formal privacy guarantee for the training process itself. These two techniques address privacy *during training*; TrustZone-based confidential inference addresses privacy *during inference*. In principle, combining all three would give an end-to-end privacy guarantee spanning both training and deployment for a fleet of edge gateways: federated learning and differential privacy protecting the training data across gateways, and confidential inference protecting each gateway's live audio and the deployed model itself.

No peer-reviewed work was found that implements and evaluates this specific three-way combination for an audio anti-spoofing model. Each primitive is individually mature, but their combination remains, at the time of writing, a proposed synthesis rather than a validated design. This is the gap RQ2 targets, and it is the reason this thesis scopes its confidential-inference proof-of-concept as a necessary first step: demonstrating that the TrustZone half of the combination is achievable end-to-end with a real anti-spoofing model is a precondition for layering federated learning and differential privacy on top of it in subsequent work, rather than something that can be assumed to work without direct verification.

3.3 From Laboratory Benchmarks to the Telephony Edge

RQ3: What deployment challenges remain in transferring audio deepfake detection from laboratory benchmarks to the residential telephony edge?

Three gaps are relevant here. The first is the generalisation gap already introduced in Section 3.1: detectors trained on curated benchmark corpora such as ASVspoof do not reliably transfer to audio recorded under uncontrolled, real-world conditions [4]. The ASVspoof challenge series itself [8] has progressively broadened its evaluation conditions in response to exactly this concern, but a gap between benchmark and deployment conditions remains an acknowledged open problem in the field.

The second is an integration gap: very few peer-reviewed studies report a com-

plete, working integration of an anti-spoofing detector with a live telephony call-handling system. Most published evaluations operate on pre-segmented recordings rather than a rolling buffer of live call audio, leaving open practical questions – window size, decision aggregation across consecutive windows, and end-to-end latency budget – that only surface once a detector is wired to a real call path. This thesis's live-call integration work (Section 4.6) is a direct, concrete attempt to close part of this gap.

The third is a demographic evaluation gap: published evaluation protocols rarely break results down by speaker demographic, even though the practical cost of a missed detection or a false alarm is not evenly distributed across the population that residential telephony fraud actually targets. This thesis's own generalisation study (Section 5.3) touches this gap directly, since one of the candidate explanatory variables investigated was speaker accent, motivated by the observation that the ASVspoof training corpus (which draws heavily on the VCTK corpus of British and Irish English speakers) may under-represent the accent diversity of a real deployment population.

3.4 Summary and Positioning

Table 3.1 summarises how the three research questions above map onto the remainder of this thesis. RQ1 is answered by the architecture selection and duration-sensitivity analysis in this chapter and applied directly in Chapter 4. RQ2 motivates the confidential-inference proof-of-concept that is this thesis's central technical contribution, described in Chapter 4 and verified in Chapter 5. RQ3 is addressed empirically by the domain-generalisation elimination study and the live-call integration work reported in Chapter 5.

RQ	How it is addressed in this thesis
RQ1	RawTFNet-16 selected on the accuracy/footprint trade-off (Section 3.1); its duration-sensitivity finding drives the sliding-window design (Section 4.6).
RQ2	No prior work combines federated learning, differential privacy, and TrustZone confidential inference for anti-spoofing; this thesis implements and verifies the confidential-inference primitive as the necessary first step (Chapters 4–5).
RQ3	The lab-to-edge generalisation gap is independently reproduced and characterised (Section 5.3); the integration gap is addressed by the live-call design and partial implementation (Section 4.6).

Table 3.1: Research questions and where each is addressed in this thesis.

Chapter 4

System Design

This chapter presents the design of the confidential-inference pipeline: the detection backbone carried over from Chapter 3, the trusted-execution architecture it runs inside, the pipeline used to convert a trained PyTorch model into a form that architecture can execute, and the partitioned deployment design that resolves the memory constraint discovered during development. Each design decision is linked explicitly back to the alternative it was chosen over.

4.1 Detection Backbone

RawTFNet-16 (Section 3.1) was used unmodified as the detection backbone: no changes were made to its topology, layer widths, or channel-width parameter $\tau = 16$. Figure 4.1 shows its three stages – a Sinc/DWS-SE-Res2Net frontend, nine Time-Frequency Convolution blocks, and a lightweight classifier – and marks the boundary at which the network was later partitioned for secure-world deployment (Section 4.4). This boundary was not introduced for this project; it already exists in the trained model's own `forward()` implementation, which is exactly `classifier(front_end(x))`.

4.2 Trusted Execution Architecture

ARM TrustZone partitions a single physical CPU into a normal world and a secure world, enforced in hardware [9]. OP-TEE [10], the open-source trusted operating system used in this project, exposes this partition through a standard programming model of two components (Figure 4.2): a *trusted application* (TA), signed C code executing inside the secure world with a small, fixed secure-memory heap; and a *client application* (CA), an ordinary normal-world Linux process that opens

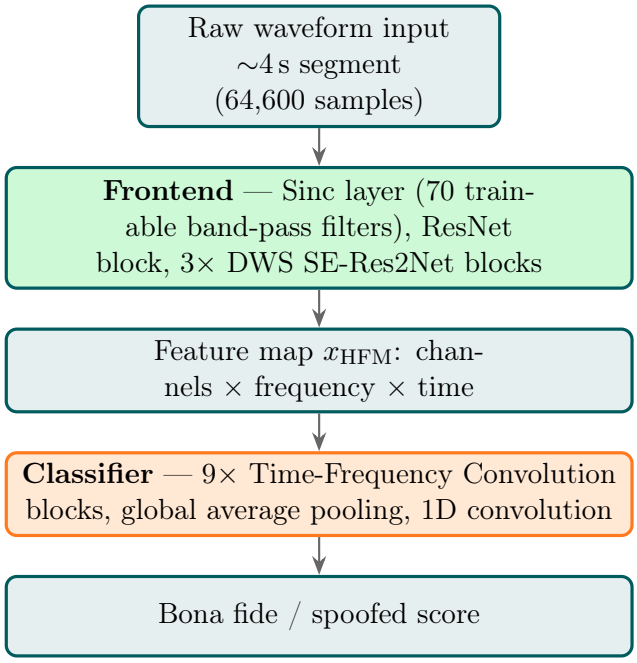


Figure 4.1: RawTFNet-16 architecture, used unmodified as the detection backbone. The frontend/classifier boundary (green/orange) is the seam used for secure-world partitioning in Section 4.4.

a session with the TA and invokes commands through the GlobalPlatform TEE Client API.

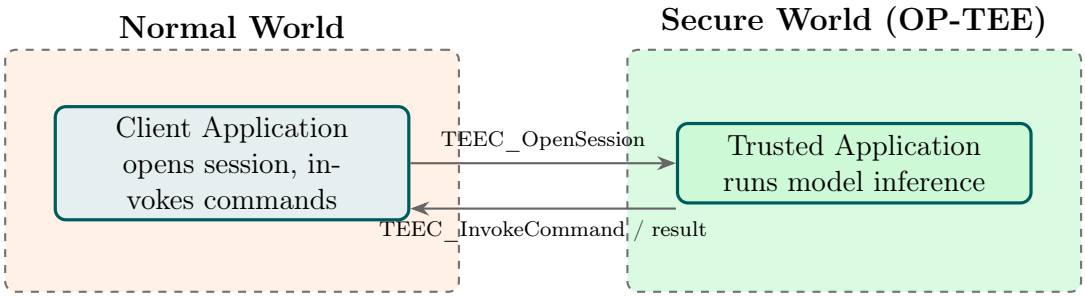


Figure 4.2: The trusted application / client application session pattern used by OP-TEE. This pattern was validated with a minimal official example (Milestone M2, Table 2.1) before any model-specific code was written.

The intended deployment target for this architecture is a Raspberry Pi 4. No physical Pi 4 was available during this phase of the work, so the entire pipeline described in this chapter was built and verified on an emulated platform instead.

4.2.1 Design decision: emulation platform

There is no mainstream, officially maintained emulator that boots a Raspberry Pi 4 with working TrustZone/OP-TEE support, because Broadcom has never published the secure-boot and TrustZone configuration details for the Pi 4's BCM2711 system-on-chip. The platform used instead is the QEMU `virt` Armv8-A reference board, the standard substitute used throughout the OP-TEE research and tooling ecosystem. It shares the same CPU architecture family (ARMv8-A / AArch64) and the same secure/normal-world security model as the Pi 4's Cortex-A72, but does not model the Pi 4's actual core microarchitecture, cache hierarchy, or memory bandwidth, and runs under full software instruction translation (QEMU TCG) rather than hardware-accelerated virtualisation. QEMU/OP-TEE is therefore treated in this thesis as a *functional* proxy for TrustZone development – demonstrating that confidential inference works and validating the trusted-application architecture – rather than a *performance* proxy; any timing figures obtained on this platform are reported as such and are not claimed to be representative of Pi 4 performance.

4.3 Model Deployment Pipeline

A trusted application cannot host a full deep-learning framework: PyTorch, ONNX Runtime, and TensorFlow all assume a POSIX-like environment with dynamic memory, threads, and a large heap, none of which is available inside a TA. Two deployment strategies were considered. Hand-porting RawTFNet's layers directly to C was rejected as high-effort and high-risk, since every convolution, the Sinc layer, and the squeeze-and-excitation modules would need independent reimplementation and verification. The alternative adopted instead exports the trained model to ONNX – a framework-neutral graph representation – and compiles the verified graph ahead-of-time to dependency-free C, checking numerical correctness at every stage against the original PyTorch output (Figure 4.3).

4.3.1 Design decision: ONNX-to-C code generation tool

Two ONNX-to-C code generators were evaluated for the final compilation step. `onnx2c` was tried first and rejected after three consecutive operator-support gaps: an unresolved external-data tensor reference, an unsupported `training_mode` attribute on batch-normalisation nodes, and an unsupported `Squeeze` pattern inside the squeeze-and-excitation module that could not be worked around. `emx-onnx-cgen` was adopted instead: it reports over 99% coverage of the ONNX opset-26 operator set, generates static C, and ships a built-in verification command that checks the generated C against ONNX Runtime with explicit numerical tolerance reporting

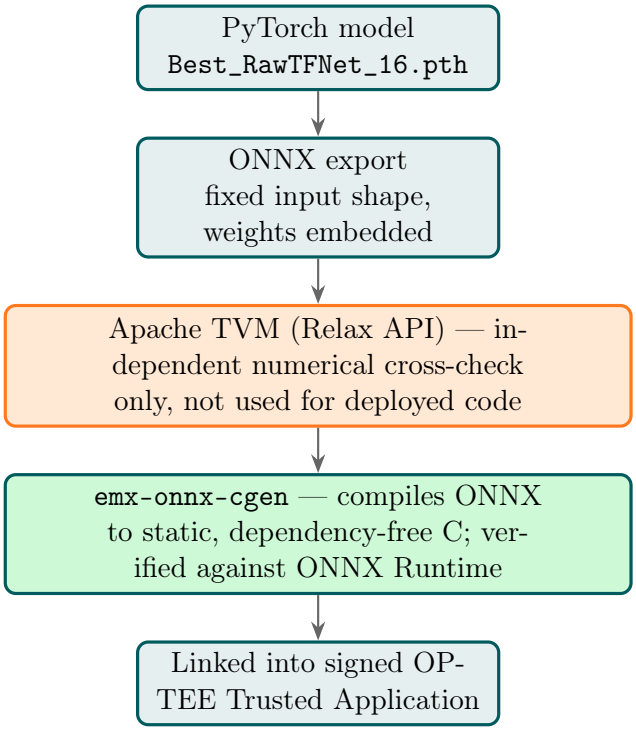


Figure 4.3: Model deployment pipeline. Correctness is checked numerically at every stage so that any discrepancy is caught at the stage that introduced it, rather than after secure-world integration.

(used throughout Chapter 5 as the correctness gate before any code was linked into a trusted application).

4.4 Resolving the Secure-World Memory Constraint

The trusted application's secure-memory pool on the emulation platform is fixed at approximately 16 MB, independent of what any individual trusted application declares. The full compiled model (frontend and classifier together) requires 319.5 MB of working memory across 191 intermediate tensors – roughly twenty times the available pool – and so cannot be loaded as a single trusted application regardless of how its build-time memory parameters are tuned. The diagnostic process that established this figure is presented as an evaluation of the initial design in Section 5.2; this section presents the resulting design.

The resolution partitions the network at the frontend/classifier boundary already present in the trained model (Figure 4.1) and deploys only the classifier inside the trusted application, running the larger frontend in the normal world in-

stead (Figure 4.4). This is architecturally related to the layer-selection partitioning used by DarkneTZ [5] (Section 3.2), though independently motivated: DarkneTZ partitions primarily to protect late-stage feature confidentiality within an otherwise generous compute budget, whereas this design partitions primarily to satisfy a hard secure-memory ceiling while still protecting the entire decision-producing stage of the network.

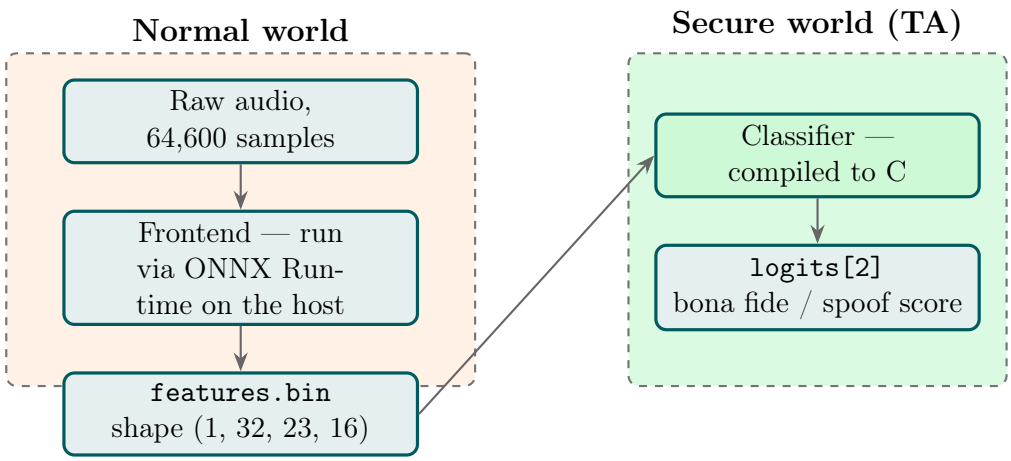


Figure 4.4: Partitioned deployment. Only the classifier's forward pass executes inside the secure world; frontend feature extraction remains in the normal world for this proof-of-concept, a scope boundary discussed further in Section 5.5.

4.5 End-to-End System Architecture

Figure 4.5 combines the preceding design elements into the full target pipeline: from a live call, through windowing and frontend feature extraction in the normal world, across the confidential-inference boundary into the secure-world classifier, and out to a live decision.

As Figure 4.5 shows, a call handled by Asterisk is captured into a rolling audio buffer, and the most recent four-second window is passed through frontend feature extraction in the normal world. The resulting features cross the confidential-inference boundary into the secure-world classifier, which returns a per-window bona fide/spoof score to the decision logic that drives the live dashboard and alert.

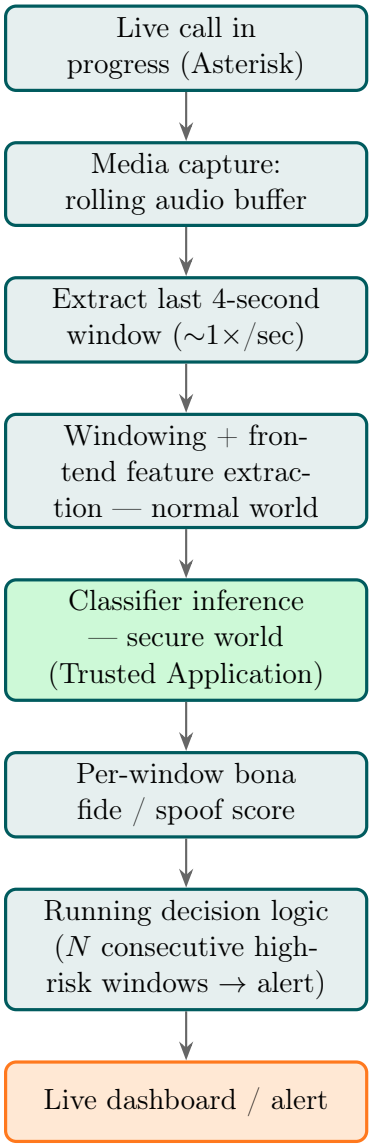


Figure 4.5: End-to-end system architecture.

4.6 Live Telephony Integration Design

The live-detection design uses a sliding window rather than a record-then-analyse approach: as audio streams from a call, the most recent four seconds of buffered audio – matching RawTFNet's native input length and its duration-sensitivity optimum (Section 3.1) – are extracted approximately once per second and passed through the pipeline in Figure 4.5. A running decision rule (raising an alert only after several consecutive high-risk windows) prevents a single noisy window from

triggering a false alarm mid-call, the same overlapping-chunk pattern used by live captioning and live-translation systems.

For this proof-of-concept, Asterisk was configured with a SIP endpoint and a dialplan extension that answers a call and records it continuously to a growing file; a host-side script polls this growing recording and extracts each new window as it becomes available, in place of a fully streaming capture path (Asterisk's AudioSocket or REST-interface external-media facilities). This is recorded as a stated limitation and near-term next step in Section 5.5 and Chapter 6.

Chapter 5

System Evaluation

This chapter evaluates the system against the three objectives set out in Chapter 1. Section 5.1 reports the numerical verification evidence gathered at every stage of the deployment pipeline. Section 5.2 presents, as a case study in the project's engineering process, the diagnosis of a significant memory-capacity failure that the initial (unpartitioned) design encountered, and how the partitioned design in Section 4.4 was arrived at. Section 5.3 reports the domain-generalisation investigation. Section 5.4 reports the status of the live telephony integration. Section 5.5 discusses the system's limitations and threats to validity.

5.1 Verification of Objective O2: Correctness of Confidential Inference

Table 5.1 summarises every point in the pipeline where a numerical or behavioural claim was independently checked rather than assumed. Each row was obtained on a held-out ASVspoof 2021 LA evaluation utterance unless stated otherwise.

Two aspects of this evidence base are worth highlighting against Objective O2. First, the end-to-end numerical agreement (six decimal places between the original PyTorch model and the secure-world trusted application) demonstrates that partitioning the network (Section 4.4) introduced no measurable accuracy cost. Second, the secure-world execution proof directly satisfies the requirement, identified during this project, that a reviewer needs independent evidence that computation genuinely occurred inside the secure world on a specific, identifiable input, not merely a correct-looking output from an otherwise unverifiable process.

Checkpoint	Evidence
OP-TEE environment	Official test suite (`xtest`): 0 failures across 41,482 subtests, 145 test cases.
TA/CA communication	Official minimal example: a test value was correctly incremented from 42 to 43 across the secure-world boundary.
PyTorch → ONNX	Scores matched to four decimal places (PyTorch −5.6216, ONNX −5.6216).
ONNX → TVM (cross-check)	Independent Apache TVM (Relax API) execution: −5.621588, matching both prior stages.
ONNX → C	Built-in `emx-onnx-cgen` verification against ONNX Runtime: maximum deviation of 7 units in the last place.
Frontend/classifier split	Classifier-only output identical to the full model's forward pass: [5.391142, −5.6215887] in both cases.
In-enclave execution	Secure-world trusted application returned [5.391142, −5.621588], matching the PyTorch reference to six decimal places.
Generalisation to a second file	PyTorch −4.9850 versus secure-world −4.985014 on an independent evaluation file.
Secure-world execution proof	Audio filename and computed score logged from inside the secure world via a console the normal-world client application cannot write to.

Table 5.1: Verification evidence at every stage of the deployment pipeline.

5.2 Diagnosing the Secure-World Memory Constraint

The partitioned design presented in Section 4.4 was not the first design attempted. The initial design compiled the full model (frontend and classifier together) into a single trusted application; this section presents, as a case study in critical evaluation of that initial design, how its failure was diagnosed and how the diagnosis led to the partitioned design.

5.2.1 The failure

The first attempt to load the full-model trusted application failed at the loader stage, before any model code executed, with the secure-world log reporting a failure in `sys_map_zi` – the loader step that maps a trusted application's zero-initialised memory into the secure world – and the client application reporting `TEEC_OpenSession failed 0xffff000c`. This indicated that the declared mem-

ory footprint could not be satisfied by the loader; the question was why, and by how much.

5.2.2 Elimination and measurement

Table 5.2 summarises the diagnostic sequence. The investigation proceeded by elimination: a deliberately trivial trusted application (with no model code at all) was used to confirm the communication pattern itself was not at fault; shortening the input window was considered and rejected, both because it breaks the network's pooling operations structurally (Section 3.1) and because it would independently degrade accuracy per the duration-sensitivity finding; and moving large buffers from static to stack allocation resolved the loader failure but produced a new, different failure (a runtime stack overflow), indicating that the true memory requirement had merely been relocated, not reduced.

Step	Finding
1	Loader failure (`sys_map_zi`). Tuning the trusted application's declared data and stack sizes individually did not resolve it.
2	A trivial, model-free trusted application successfully exchanged a full audio buffer across the secure-world boundary, isolating the fault to the model's own memory requirement rather than the communication pattern.
3	Shortening the input window was rejected: it breaks the network's internal pooling operations structurally and would independently degrade accuracy.
4	Relocating large buffers to the runtime stack resolved the loader failure but produced a runtime stack-overflow panic, indicating the requirement had been relocated, not reduced.
5	Direct measurement (summing every allocation call in the generated source) established the true total: 319.5 MB across 191 workspace tensors, against a secure-world pool of approximately 16 MB – conclusively ruling out any parameter-tuning solution.
6	*Resolution:* partition the model at its existing frontend/classifier boundary (Section 4.4) and deploy only the classifier, whose workspace requirement is several orders of magnitude smaller.

Table 5.2: Diagnostic sequence leading to the partitioned design.

5.2.3 A self-corrected diagnostic error

One point in this investigation is reported in detail because it illustrates a diagnostic discipline rather than a purely technical fix. During Step 4, the trusted application occasionally returned plausible-looking partial scores instead of either a correct result or an outright failure. The initial hypothesis was that these values reflected partial computation, as though the model were tracking how far it had progressed before exhausting available heap. This hypothesis was incorrect, and was caught by reading the generated source directly rather than trusting the superficially plausible pattern in the output: the compiled model allocates its intermediate tensors through a long sequence of individual allocation calls and, by design, returns silently without writing to its output buffer if any single allocation fails. The values observed were uninitialised memory already present in the client application's result buffer, not a signal from the model. Once identified, the trusted-application handler was changed to zero-initialise its output buffer before invoking the model, so that any future silent allocation failure would produce an unambiguous, exact-zero result rather than a plausible but meaningless one. This correction is reported here because verifying an unexpected result against its actual source, rather than accepting a plausible narrative for it, is precisely the discipline that Section 4.3's stage-by-stage verification approach was designed to enforce.

5.3 Domain-Generalisation Investigation

Real-world audio – including a genuine bona fide recording of the researcher's own voice, a text-to-speech spoof, and independent test recordings later supplied by the project supervisor – consistently misclassified as spoofed through the verified pipeline, including recordings that were genuinely bona fide. Because the pipeline's correctness had already been established (Section 5.1), this was investigated as an empirical question about the trained model's generalisation, using the controlled elimination methodology described in Section 2.2.2.

With every readily fixable, single preprocessing confound eliminated or corrected, the residual misclassification is attributed to the trained model keying on synthesis artefacts specific to the ASVspoof training distribution rather than to general spoof characteristics – a genuine domain-generalisation limitation consistent with the gap identified independently by Müller et al. [4] and discussed in Section 3.3. Independent test recordings later supplied by the project supervisor refined rather than reversed this conclusion: one clip scored close to the decision boundary (bona fide 0.54 versus spoof -0.74) rather than in the confidently misclassified range observed for earlier recordings, suggesting the domain gap is better

Candidate cause	Finding
Sample rate	Already handled correctly by the existing preprocessing pipeline; ruled out.
Loudness / level mismatch	A real, confirmed confound: the researcher's recording was approximately 12 dB quieter than ASVspoof reference audio and was not peak-normalised. Peak normalisation was added to the preprocessing pipeline. Real-world audio still misclassified after this fix, so loudness was confirmed as a real but partial cause.
Speaker accent	A same-pipeline recording from a native Irish speaker (matching the accent distribution of the ASVspoof training corpus) misclassified identically to non-native-accented recordings, weakening accent as the primary explanation.
Background noise floor	A denoise preprocessing step moved scores only marginally and did not change the classification outcome on either test file; ruled out.

Table 5.3: Domain-generalisation elimination study: candidate causes of real-world misclassification.

described as a spectrum of severity across recordings than a uniform failure.

This finding is reported as a characterised limitation of the trained detector rather than a defect in the confidential-inference system under evaluation: the pipeline's own correctness was independently established in Section 5.1 before this investigation began, and the misclassification pattern was reproduced consistently across every recording tested, indicating a property of the model rather than an intermittent pipeline fault.

5.4 Live Telephony Integration Status

Against Objective O3, the live-call integration design in Section 4.6 was partially implemented and evaluated. Asterisk installation, SIP endpoint and dialplan configuration, and softphone registration were completed and confirmed: a test call was placed and the resulting recording file was observed growing in real time for the duration of the call, confirming that live call audio reaches the host environment as intended. The full watcher-loop integration that extracts each new four-second window and drives a fresh secure-world inference per window was designed and documented but had not, at the time of writing, been executed as a complete

end-to-end demonstration against a live call. This is reported as an incomplete milestone (M8 in Table 2.1) rather than a failed one, and is carried forward as immediate future work in Chapter 6.

5.5 Limitations and Threats to Validity

Limitation	Explanation
No physical target hardware	All results were obtained on the QEMU ARMv8-A emulation platform (Section 4.2.1), not a physical Raspberry Pi 4; this validates the software architecture and TrustZone isolation model, not device-specific behaviour.
No hardware root of trust	The emulated secure world is a software CPU-mode partition with no fused keys or physical secure boot chain; hardware-backed attestation is out of scope and identified as future work.
Static, compiled-in model weights	Model weights are compiled directly into the trusted application binary rather than loaded via secure storage, meaning the application would need re-signing on every retrain – a reasonable simplification for a feasibility proof-of-concept, but not workable for the longer-term federated-learning goal in Section 3.2.
Frontend stays in the normal world	Only the classifier executes inside the secure world for this proof-of-concept (Section 4.4); an attacker with normal-world root access could in principle supply manipulated features to the trusted application.
Real-world detection accuracy	The domain-generalisation gap characterised in Section 5.3 means the underlying detector's real-world accuracy is not yet established at the same confidence level as its in-distribution ASVspoof accuracy.
Live capture is not yet streaming	The current integration polls a growing recording rather than receiving a genuinely live audio stream (Section 5.4).
Small generalisation-study sample	The elimination study in Section 5.3 used a small, opportunistically gathered set of recordings rather than a large, demographically balanced corpus.

Table 5.4: Limitations and threats to validity.

Chapter 6

Conclusion

This thesis set out to design and demonstrate the confidential-inference component of a privacy-preserving edge gateway for real-time audio deepfake detection in residential telephony, addressing three objectives: selecting and justifying a detection architecture suited to resource-constrained edge deployment, implementing and verifying a confidential-inference pipeline for that architecture inside an ARM TrustZone secure world, and characterising the system's behaviour under conditions representative of a real telephony deployment.

Objectives O1 and O2 were met in full, and O3 was met in part. RawTFNet-16 was selected and justified against the accuracy/footprint trade-off central to Research Question 1, and its published duration-sensitivity behaviour was carried directly into the live-detection window design. The confidential-inference pipeline was implemented, and its correctness was independently verified at every stage from the original PyTorch model through to in-enclave execution, matching the reference implementation to six decimal places – directly answering Research Question 2's requirement that the TrustZone half of a combined privacy-preserving design be shown to work before federated learning and differential privacy are layered on top of it. The system's behaviour under real deployment conditions was characterised on two fronts: a controlled elimination study established that the trained detector's real-world misclassification is a genuine domain-generalisation limitation rather than a correctable preprocessing artefact, directly reproducing and extending a gap identified independently in the wider literature (Research Question 3); and a live telephony integration was designed, and partially implemented and evaluated, against Asterisk.

The most significant engineering obstacle encountered – a secure-world memory capacity failure that no amount of parameter tuning could resolve – was not sidestepped but measured precisely (319.5 MB of required workspace against a $\sim$16 MB secure-memory pool) and resolved architecturally, by partitioning the network at a seam that already existed in its own trained structure. This is, to the

best of the author's knowledge, a first working demonstration that a confidential-inference architecture for a real audio anti-spoofing network can be made to fit a memory-constrained TrustZone enclave without any loss of numerical accuracy.

6.1 Future Work

Four directions follow directly from the limitations identified in Section 5.5. First, the pipeline should be benchmarked on physical Raspberry Pi 4 hardware once available, both to obtain representative latency figures and to validate the confidential-inference design against a hardware root of trust rather than an emulated one. Second, the live-call integration should be completed end-to-end and migrated from polling a growing recording to a genuinely streaming capture path. Third, a larger and demographically structured audio corpus should be used to extend the domain-generalisation study in Section 5.3 into a quantitative accuracy figure, alongside domain adaptation or fine-tuning of the detector on real-world and synthetic speech to close the gap the study identified. Finally, and most directly continuing the motivation set out in Chapter 1, the federated learning and differential privacy components of the full thesis scope should be layered onto the confidential-inference primitive verified here, to realise the complete end-to-end privacy guarantee that Research Question 2 identifies as an unaddressed gap in the literature.

The work reported in this thesis establishes that the hardest architectural question – whether a real anti-spoofing model can run, correctly and verifiably, inside a memory-constrained trusted execution environment – has a demonstrated, positive answer. What remains is extending that verified foundation to a complete, streaming, hardware-validated deployment.

Bibliography

[1] Y. Xiao, T. Dang, and R. K. Das. Rawtfnet: A lightweight cnn architecture for speech anti-spoofing. In *2025 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, 2025.

[2] Hemlata Tak, Jose Patino, Massimiliano Todisco, Andreas Nautsch, Nicholas Evans, and Anthony Larcher. End-to-end anti-spoofing with RawNet2. In *ICASSP 2021 – 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6369–6373. IEEE, 2021.

[3] Jee-weon Jung, Hee-Soo Heo, Hemlata Tak, Hye-jin Shim, Joon Son Chung, Bong-Jin Lee, Ha-Jin Yu, and Nicholas Evans. AASIST: Audio anti-spoofing using integrated spectro-temporal graph attention networks. In *ICASSP 2022 – 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6367–6371. IEEE, 2022.

[4] Nicolas M. Müller, Pavel Czempin, Franziska Dieckmann, Adam Froghyar, and Konstantin Böttinger. Does audio deepfake detection generalize? In *Proceedings of Interspeech 2022*, pages 2783–2787, 2022.

[5] Fan Mo, Ali Shahin Shamsabadi, Kleomenis Katevas, Soteris Demetriou, Ilias Leontiadis, Andrea Cavallaro, and Hamed Haddadi. DarkneTZ: towards model privacy at the edge using trusted execution environments. In *Proceedings of the 18th International Conference on Mobile Systems, Applications, and Services (MobiSys)*, pages 161–174, 2020.

[6] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1273–1282, 2017.

[7] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 308–318, 2016.

[8] Junichi Yamagishi, Xin Wang, Massimiliano Todisco, Md Sahidullah, Jose Patino, Andreas Nautsch, Xuechen Liu, Kong Aik Lee, Tomi Kinnunen, Nicholas Evans, and Hector Delgado. ASVspoof 2021: accelerating progress in spoofed and deepfake speech detection. In *Proc. 2021 Edition of the Automatic Speaker Verification and Spoofing Countermeasures Challenge (ASVspoof 2021)*, pages 47–54, 2021.

[9] ARM Limited. ARM security technology: Building a secure system using TrustZone technology. ARM White Paper, PRD29-GENC-009492C, 2009.

[10] Linaro OP-TEE Project. OP-TEE documentation. `https://optee.readthedocs.io`, 2024. Accessed 2026.

Appendix A

Source Code Repository

A.1 Repository link

The complete source code, build and run instructions, and supporting resources for this project are published in the following public GitHub repository:

`https://github.com/waleed0537/masters-project`

The repository `README` describes the project, lists its features, and gives step-by-step instructions for building, deploying and running the system at three levels: checking the model and the generated C classifier, running the classifier inside the emulated secure world, and running it on live telephone calls. Following these instructions reproduces the results reported in Chapter 5.

A.2 Repository contents

The repository (default branch `main`) is organised as follows.

- `rawtfnet/` — the RawTFNet implementation, the trained checkpoint (`Best_RawTFNet_16.pth`, in `ckpts/`), and the export, test and equal-error-rate scripts (`export_*.py`, `test_audio.py`, `compute_eer.py`);
- `artifacts/` — the exported ONNX models (frontend, classifier and full model) and the reference feature file `features.bin`;
- `scripts/` — audio preparation (`prepare_audio.py`), model cleaning and C generation, the Apache TVM cross-check, and the native check of the generated C;

- `secure-world/` — the OP-TEE Trusted Application and client application (including the generated classifier `rawtfnet_model.c` and the minimal maths subset `ta_math.c`), the native test harnesses, the QEMU launch script and the pinned OP-TEE manifest;
- `live-call/` — the Asterisk PJSIP and dialplan configuration, `live_detect.py`, and the guest watcher script (Section 4.6);
- `results/` — the unedited log of the live-call session and a summary of the results;
- `docs/` — progress reports and design notes;
- `thesis/` — this thesis in PDF format;
- `screencast/` — the screencast demonstrating the application working;
- `README.md` — project overview, feature list, and step-by-step instructions for building, deploying and running the system.

A.3 Notes for reproduction

- **Datasets.** The ASVspoof 2021 LA evaluation data is not redistributed in the repository because of its size and licence; the `README` gives the download location.
- **Environment.** Model work (ONNX export, frontend execution, C generation) uses a dedicated Python virtual environment. The OP-TEE trusted-application build must be run *outside* that environment, because trusted-application signing requires a system Python installation that provides the `cryptography` module.
- **Platform.** All secure-world results were obtained on the QEMU Armv8-A emulation of OP-TEE (Section 4.2.1), running in an Ubuntu virtual machine with bridged networking so that Asterisk SIP and RTP traffic is directly reachable.